Accessibility Testing Framework using Playwright + axe-core

1. Purpose

The goal of this project is to establish an automated accessibility testing framework that ensures all web pages meet WCAG 2.1 AA standards.
The framework will use Playwright for browser automation and dynamic user interactions, and axe-core for accessibility rule evaluation.

2. Scope

In Scope:
- Automated accessibility testing for web applications.
- Coverage of WCAG 2.1 A/AA guidelines.
- Integration of Playwright + axe-core into local environments and CI/CD pipelines.
- Reporting of violations with detailed descriptions and impacted elements.

Out of Scope:
- Manual accessibility audits (e.g., screen reader usability).
- Automated color contrast verification beyond axe-core's capability.
- Accessibility tests for non-web platforms (mobile, desktop).

3. Objectives and Success Criteria

- Detect accessibility violations automatically.
- Ensure WCAG 2.1 AA compliance.
- Enable developers to fix issues early.
- Support a scalable, maintainable test framework.

4. Functional Requirements

4.1 Test Execution: Use Playwright to navigate, simulate user interactions, and trigger axe-core.
4.2 Accessibility Scanning: axe-core should report rule ID, WCAG reference, severity, and element selector.
4.3 Test Reporting: Log violations and export results to JSON/HTML.
4.4 Configurability: URLs and rule sets configurable.
4.5 CI Integration: Run tests automatically on PRs and merges.

5. Non-Functional Requirements

- Performance: Scan page within 10s total.
- Scalability: Support 50+ pages per run.
- Maintainability: Simple structure for adding pages.
- Usability: Clear reports for developers and QA.
- Reliability: Consistent results across browsers and CI.

6. Technical Stack

- Playwright: Browser automation
- axe-core: Accessibility rule engine
- @axe-core/playwright: Integration bridge
- Node.js: Runtime
- npm/yarn: Dependency management
- GitHub Actions/Jenkins/GitLab CI: CI/CD integration

- Optional: Allure or HTML report generator

7. Implementation Plan

Step 1: Setup environment with npm install commands.
Step 2: Create test file (accessibility.spec.js) using Playwright + axe-core.
Step 3: Run tests via 'npx playwright test'.
Step 4: Generate JSON/HTML reports.
Step 5: Integrate with CI/CD (example GitHub Actions workflow).

8. Deliverables

- Working Playwright + axe-core test suite.
- Configurable URL list.
- CI-integrated pipeline.
- JSON/HTML reports.
- Developer documentation.

9. Risks & Mitigations

- False positives → Allow configurable exclusions.
- Dynamic content not ready → Use wait conditions.
- Test flakiness → Use deterministic mocks.
- Developer resistance → Gradual enforcement.

10. Future Enhancements

- Visual HTML reports.
- Storybook integration.
- Slack/Teams notifications.
- Dashboard for aggregated reports.